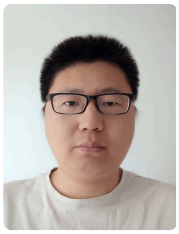
# 张恒

**求职意向：嵌入式开发工程师**


工作年限：16 年
期望城市：青岛
电　　话：151-9248-5800
邮　　箱：354379271@qq.com

## 教育背景

**大连理工大学（985）** | 软件工程（嵌入式方向） | 本科 | 2006.09 – 2010.06 | 英语六级

## 个人优势

- 16 年 C/C++ 开发经验，其中 5 年 ARM/VxWorks/FreeRTOS 嵌入式网络协议栈开发，11 年 Linux 网络通信开发
- 熟悉 TCP/IP、HTTP、USB、IPP、AirPrint 等通信协议，主导 AirPrint 与 IPP over USB 开发并通过苹果官方认证
- 擅长嵌入式实时系统与 Linux 系统的性能调优（网络、CPU、内存）和复杂问题根因定位
- 长期支持北美、欧洲运营商，英语可作为工作语言，多次牵头跨团队新功能开发与技术攻关
- 持续关注新技术，具备大模型推理优化与 AI Agent 应用的开发实践

## 专业技能

| | |
|---|---|
| **编程语言** | 熟悉 C/C++ 及现代 C++ 特性；熟练使用 Python、Golang、Shell |
| **嵌入式** | 熟悉 ARM 平台与 VxWorks、FreeRTOS 实时操作系统，掌握多任务调度、消息队列、信号量与优先级继承；会用 WindView 分析多任务性能 |
| **接口外设** | 熟悉 STM32 平台 USB 2.0 全速设备端开发（打印机类、批量传输），有量产经验；熟悉 UART、I2C、SPI 总线与 GPIO、DMA 外设 |
| **网络协议** | 依据 RFC 在嵌入式设备上实现 HTTP、UPnP、SNMP、IPP/Bonjour 等协议栈；熟悉 TCP/IP、SCTP 与 Diameter |
| **调试调优** | 熟悉 Linux 多线程与 epoll 高并发编程；熟练使用 gdb、perf、tcpdump、Wireshark、Valgrind 等工具调试与性能分析 |
| **云原生与 AI** | 熟悉 Docker、Kubernetes 容器化部署；用 C++ 从零实现 Transformer 推理，开发过 ReAct Agent |

## 工作经历

### 诺基亚通信投资（中国）有限公司青岛分公司 · 高级软件工程师　2015.06 – 至今

| | |
|---|---|
| **高性能后端** | 负责 4G/5G 用户数据库（HSS/UDM）Linux C++ 后端，基于 epoll、多线程与共享内存实现 Diameter、HTTP/2 高并发低时延处理 |
| **模块负责** | 网络通信、数据库缓存、通用应用中间件 3 个核心模块负责人，负责架构设计、代码评审与质量把关 |
| **云原生改造** | 主导用户数据库（HSS）后端服务器的 Docker/Kubernetes 容器化改造，负责全产品不中断升级方案设计 |
| **客户支持** | 为北美、欧洲运营商定位并解决核心网现网的网络、CPU、内存泄漏等重大问题（2018.10 – 2020.09） |
| **团队带领** | 多次牵头新功能开发，带领 6～7 人 Scrum 小组从设计到交付；负责新员工培训与技术分享 |

### 日冲信息（大连）有限公司 · 嵌入式软件工程师　2010.06 – 2015.05

| | |
|---|---|
| **协议及驱动开发** | 负责打印机网络协议栈（HTTP、SNMP 等）与 USB 等相关驱动，在 ARM/VxWorks/FreeRTOS 上用 C/C++ 实现，适配资源受限设备 |
| **功能开发** | 主导打印、扫描等功能开发（AirPrint、IPP over USB、谷歌云打印），产品连续通过苹果 AirPrint 认证 |
| **性能优化** | 分析多任务与网络传输瓶颈，提升打印与扫描传输速率 |
| **认证测试** | 主持搭建协议认证环境，推动网络协议栈通过 IPv6 Ready Logo Phase-1/Phase-2 认证 |
| **UI开发测试** | 负责触摸屏GUI界面与内嵌Web管理网页（HTML、JavaScript）的开发，并用 Selenium 完成自动化测试 |

## 项目经历

### 苹果 AirPrint 打印协议开发与认证（ARM/VxWorks） · 主导开发 2013.11 – 2015.05

**背景** 产品线需支持 iPhone/iPad/macOS 免驱动无线打印并通过苹果官方认证，设备运行在内存不超过 256 MB、CPU 受限的 ARM/VxWorks 平台上

**我的工作**

- 服务发现：基于 Bonjour（mDNS/DNS-SD）实现打印服务的发现与通告
- 作业处理：基于 IPP 实现作业提交、状态查询与错误处理，文档数据边收边处理，支持 PDF、JPEG、URF 等格式的打印流水线
- 问题攻关：认证测试中小文档能正常打印，大 PDF 却失败或卡住；用 Wireshark 抓包发现 iOS 以 HTTP 分块传输（chunked）发送文档，而嵌入式 HTTP 服务器只支持 Content-Length，于是为其增加分块解码，边收边交给打印流水线，最终通过认证
- 认证测试：搭建认证环境，执行功能、互操作与回归测试，跟进 1.1 ~ 1.5 规范差异并修复兼容问题

**成果** 6 款打印机连续通过 AirPrint 1.1 ~ 1.5 认证，无线打印能力随产品迭代上市商用

### 基于 STM32/FreeRTOS 的打印机 IPP over USB 开发 · 主导开发与测试 2013.11 – 2013.12

**背景** 传统 USB 打印依赖主机驱动，作业状态信息少；针式打印机基于 STM32/FreeRTOS，整机内存只有 128 KB，无法照搬常规 IPP 实现做免驱动打印

**我的工作**

- 接口开发：在 STM32 片上全速 USB（12 Mbps）上配置 2 个打印机类接口，一路传作业、一路查状态，各配一对批量端点，并保留传统打印接口。解决免驱动打印与老主机兼容不能同时满足的问题
- 中断设计：USB 中断只把收到的数据写入预分配静态缓冲池，凑满一块或收到短包后经消息队列交给对应任务，并把中断优先级放在 FreeRTOS 可管理范围内。解决中断里处理过重，以及在中断中调用系统接口不安全的问题
- 流控设计：缓冲池耗尽时不再重新使能 OUT 端点，由硬件自动回 NAK 暂停主机发送，任务归还缓冲区后再恢复接收。解决仅 128 KB RAM 下接收既丢包又溢出的问题
- 报文处理：每个接口单独起一个任务，用状态机解析 Content-Length 和 chunked，报文体边收边交给打印流水线，共享作业状态用带优先级继承的互斥信号量保护。解决报文边界识别和多任务同时改作业状态的问题

**成果** 打印机在 USB 直连下实现免驱动打印与作业状态查询；随 6 款机型上市

### 美国运营商通信网络延迟故障根因定位（T-Mobile） · 攻关负责人 2019.06 – 2019.09

**背景** 运营商光纤中断后流量绕路，后端服务间 SCTP 长连接时延升至 140 ms 以上，约 229 万用户受影响，语音呼叫量下降 25%

**我的工作**

- 攻关协调：作为技术负责人牵头跨团队攻关，与美国、印度同事协作，协调客户现场支持和本地测试团队，按止损、复现、修复推进。保证现网止损和实验室复现在两地同时进行
- 业务止损：与客户一起分析日志和抓包，隔离高时延链路并迁移流量。先把业务从绕路链路移走，12 小时内恢复业务
- 故障定位：用 tcpdump、perf 对比正常与异常连接，发现发送队列堆积并出现 EAGAIN。确认单线程阻塞发送，一条慢连接拖垮同线程上的其他连接
- 根因定位：组织实验室按 140 ms 以上同等时延复现，控制变量测试网卡队列、发送缓冲区以及物理机与虚拟机。锁定 KVM 虚拟网卡多队列导致报文乱序和快速重传，高时延下拥塞窗口难以恢复
- 修复开发：用 C++ 实现连接级自动隔离，5 秒滑动窗口内累计 10 次写失败即摘除该连接，流量分到其余 23 条，10 秒后自动重连；并调大发送缓冲区、缩短 epoll 超时。解决一条慢连接阻塞整个发送线程的问题

**成果** 12 小时内恢复业务，72 小时内补丁被现网采纳；网卡改为单队列后单连接吞吐提升约 10 倍；推动建立广域网时延测试环境与链路级监控告警

### 基于共享内存的数据库缓存系统设计 · 模块负责人 2025.09 – 2025.12

**背景** 用户数据库前端上百个业务进程各自从后端数据库加载配置数据。启动与刷新时同一份配置被重复查询，数据库压力大、CPU 过载，且各进程数据不一致

**我的工作**

- 架构设计：新增独立缓存进程，统一从数据库加载配置并写入共享内存，业务进程再同步本地副本。解决上百个进程各自查库带来的数据库压力、CPU 过载和数据不一致，使查询由每进程一份变为全节点一份，查询量降低 40 倍
- 通知服务开发：用 C++ 开发 HTTP Server（epoll + 线程池）接收数据库变更通知，双 Pod 以 Active-Active 部署并兼做负载均衡。解决变更集中到达时的接入瓶颈和单点问题，处理能力达到 2500 TPS
- 一致性设计：变更到达后只重新缓存变更条目或整表，再通知业务进程刷新本地副本，避免全量重载再次打满 CPU；用周期刷新和每日定点刷新兜底，解决通知丢失造成的数据不一致
- 质量把关：主导设计与代码评审，整理共享内存缓存的排障方法，用于生产问题定位和新员工培训

**成果** 消除 CPU 过载，数据库查询量降低 40 倍；已在 T-Mobile、Verizon 商用

**后端服务容器化改造与不中断升级** · 主导开发 2020.06 – 至今

**背景** 电信核心网用户数据库由多个后端服务组成，原以虚拟机交付，升级需要 5 小时停机维护，业务中断

**我的工作**

- 容器化改造：牵头认证处理、数据变更通知等 C++ 核心服务从虚拟机迁到 Kubernetes/OpenShift，进程改为单容器多进程，配置改为 ConfigMap 注入并支持热加载。解决服务绑定虚拟机交付、配置变更必须重启进程的问题
- 优雅退出开发：用 C++ 处理 SIGTERM，摘除 Diameter 与 HTTP/2 长连接并排空在途请求，同时实现就绪/存活探针。解决滚动升级时进程被直接终止、长连接中断和在途请求丢失的问题
- 兼容设计：设计新旧版本消息与数据格式的兼容方案，使升级期间新旧版本 Pod 可以混跑。解决滚动替换时新旧实例无法同时处理业务的问题
- 控制器开发：用 Golang 独立开发基于 CRD 的 Kubernetes 控制器，管理负载均衡 Pod 的滚动更新，并参与全产品 8 个 Helm Chart 的发布编排与联调。解决负载均衡实例与业务服务无法一起滚动升级的问题

**成果** 升级由 5 小时停机窗口变为不停机滚动升级，升级期间请求成功率 99.99%，单 POD 2500 QPS；已在 T-Mobile、Verizon、Vodafone 等欧美运营商商用

---

**K8s POD CPU 性能不均根因分析与调优** · 主导分析与调优 2025.09 – 2025.10

**背景** 新版本容器平台压测中，同一节点上 8 个业务 Pod 承载相同流量，CPU 占用却相差600m，容量评估失真，影响交付

**我的工作**

- 原因排除：用负载均衡计数和监控数据逐项复核，确认 8 个 Pod 流量一致、监控读数可信。排除流量分发不均和监控误差，避免后续分析建在错误前提上
- 访存分析：用 numactl、Intel PCM 量化跨 NUMA 访存，并用 Kubernetes 拓扑管理器把 Pod 约束到单个 NUMA 做对照。证明跨 NUMA 访存不是 CPU 差异的主因
- 根因定位：设计十余组控制变量测试，逐项比较 Turbo 睿频、中断亲和、电源策略和 Pod 分布。定位主因是 kubelet 的 CPU 分配使 Pod 在两个 NUMA 上分布不均，高负载节点睿频更低（2.6 vs 2.9 GHz），网卡中断集中进一步拉大差异；中断亲和本身只贡献约 40 ~ 50m
- 方案设计：建议负载均衡按 Pod CPU 使用率分发流量，向 Kubernetes 社区提交 Issue 反馈 kubelet 的分配问题，并评估预留 CPU、关闭睿频等方案的代价。解决容量评估缺少可落地依据的问题

**成果** 平衡两个 NUMA 的负载后，Pod 间 CPU 差异由约 600m 降至 150m；关闭 Turbo 可降至 100m 以内，但性能下降 10% ~ 15%，为容量规划提供量化依据

---

**C++20 大模型 CPU 推理引擎** · 个人项目 2026.04 – 2026.06

**背景** 在无 GPU 的笔记本 CPU 上从零实现 GPT-2，不依赖推理框架

**我的工作**

- 矩阵乘优化：循环重排、三层分块、AVX2 与 FMA 微内核加多线程，从 0.6 提升到 157.3 GFLOPS（274 倍，达峰值 66%）
- 解码优化：KV Cache 零拷贝，按列并行的矩阵向量乘配合常驻线程池动态抢块，解决大小核上线程越多越慢的问题
- 正确性验证：两套前向路径互校，并以 numpy 逐层比对，126 个用例通过 ASan、UBSan、TSan

**成果** 解码速度由 7.5 提升到约 111 token/s（约 15 倍）

---

**AI 智能运维终端** · 独立开发 2026.07 – 2026.07

**背景** 运维排障要在多台主机间反复敲命令，历史工单里的处理经验也难以在现场复用

**我的工作**

- 终端开发：基于 Electron 与 TypeScript 开发内置 AI 侧边栏的 SSH 终端，用 Function Calling 实现 ReAct Agent，接入 30 余个运维工具，危险命令需人工审批后才执行。解决多台主机间反复敲命令，以及危险命令被直接执行的问题
- 上下文工程：按当前工具集裁剪提示词以命中前缀缓存，对话超出预算时自动压缩记忆；技能（Skills）按需加载，做渐进式披露。多主机任务交给只读子 Agent 分头排查，只汇总结论。解决长对话占满上下文窗口，以及技能一次全部加载导致提示词膨胀的问题
- 检索开发：增量同步 Jira 工单，用内部 Ollama 部署的千问 Embedding 向量化后存入 Chroma，检索时向量与 BM25 双路召回并用 RRF 融合排序。解决排障时难以复用历史工单经验的问题

**成果** 获诺基亚内部 AI 创新大赛二等奖

## 专利、荣誉与证书

**专利** 第一发明人，PCT 国际专利 WO2025175539A1（《基于设备信息的 AKMA 认证》）

**荣誉** AI 智能运维终端获 2026 诺基亚 AI 创新大赛二等奖；2021.10 因绩效突出获授 1000 股股权激励

**软考** 软件设计师（中级）

**红帽认证** OpenShift 应用开发者（证书编号 240-240-408）